

Report

Part 1

We were asked to develop 3 java classes which are Vector, Record and TestRecord (which contains the main method to see if the implemented code works as intended). The class Vector contains only 1 attribute elems which is a list of double values describing the vector. There are 11 methods in this class: getDim, add, subtract, sqrt, dotProduct, norm, toString, multiply and divide (last 2 use polymorphism, so depending on the arguments either one or another method is used). Class Record has 2 attributes which are input and output of types Vector and double, respectively. There are 3 methods: getInput, getOutput, toString. Last class, TestRecord, is the main function which just uses other 2 classes to check their functionality. Methods names are self-explanative, the only 2 methods to highlight are multiply and divide in Vector class. They are identical, except for the operation. If the argument is of type Vector, then the operation is performed element-wise, if the argument is of type double, then every element is multiplied/divided by that scalar.

Part 2

The only alternative solution we thought of is the constructor method for Vector class. We could ask for the dimension of the vector, initialise with zeros and then, with a setter method, fill the array in. But, at the end we thought that the values of the array should be already known, as otherwise what would be the reason to initialise an instance of class Vector to operate with its values if they are not known. Therefore, we chose the solution we implemented as it is the fastest and most efficient one. As described above, we used polymorphism for multiply and divide methods in class Vector as the idea of multiplication/division using a vector or scalar is very similar, therefore, to not confuse ourselves with the names of the methods, we decided to use just one.

Part 3

When it comes to testing the correct implementation of the classes, after creating user-friendly instances (i.e. instances with values for easy computations), it prints the method to be tested, the expected solution and the actual value(s) returned by each of the methods. Thus, the user only needs to compare the values on the left with the ones on the right to prove the correct implementation of the classes, which, indeed, were all correctly implemented. Finally, as a minor issue that was presented to us during the test development, it consisted in the fact that, since most of the methods modify the actual values of the instances, we firstly thought that the methods were not working as expected, but it was actually that we were considering the initial values of the instances to compute the expected results, and not the ones returned by the previous method.